Deduced return type for overriding functions via `auto`

Document #:	DnnnnR0
Date:	2026-06-27
Project:	Programming Language C++
Audience:	SG17 (EWG Incubator) EWG (Evolution Working Group)

Reply-to:	Alex Tsvetanov <alex_tsvetanov_2002@abv.bg>

Contents

1 Abstract

A function that overrides a virtual function must, today, repeat the overridden function's return type — either by spelling it out or by reconstructing it with a trailing-return-type such as `-> decltype(std::declval<Base>().f())`. This paper proposes that a function declared with the bare placeholder `auto` *and* the `override` specifier deduce its return type to be **exactly the return type of the function it overrides**. Because `override` already guarantees that a matching base function exists, the deduction is unambiguous, requires no function body, and the resulting function is an ordinary virtual function in every other respect.

Before

```
struct A {
  virtual int test() { return 5; }
};

struct B : A {
  // Return type must be restated, either lit
  int test() override { return 6; }

  // ...or reconstructed when A::test's type
  auto test2() -> decltype(std::declval<A>().t
      override { return 6; }
};
```

After

```
struct A {
  virtual int test() { return 5; }
};

struct B : A {
  // Return type is taken from the overridden function.
  auto test() override { return 6; }

  auto test2() override { return 6; }
};
```

2 Introduction

C++ requires an overriding virtual function to repeat the return type of the function it overrides, even though that type is fixed by the language and carries no new information. This paper proposes a small, self-contained extension: a function declared with the bare placeholder `auto` together with the `override` specifier takes its return type directly from the function it overrides. The change is a pure language extension — it makes a presently ill-formed spelling well-formed, alters no existing program, and requires no library, ABI, or object-model changes. The remainder of the paper motivates the feature, discusses the design, and provides proposed wording.

3 Revision history

R0

— Initial revision. The return type is taken from the overridden function and fixed at the declaration; each `return` statement must already yield that exact type, with no implicit conversion ([design.returns]); `auto` and the written-out return type may be mixed across the in-class declaration and an out-of-line definition ([design.outofline]).

4 Motivation and scope

4.1 The problem

When overriding a virtual function, the override's return type is constrained by the language: it must be *identical* to the base function's return type, or a legal covariant type ([class.virtual]). The programmer therefore gains nothing by restating the return type — it carries no new information, it cannot legally differ (except for covariance), and it must be kept in sync by hand.

This redundancy becomes actively harmful when the base return type is verbose, dependent, or obtained through metaprogramming. The idiomatic way to "inherit" the return type today is:

```
struct B : A {
  auto test() -> decltype(std::declval<A>().test()) override { return 6; }
};
```

That trailing-return-type is pure boilerplate: it names the base class explicitly (`A`), it duplicates the function name (`test`), and it is fragile under refactoring — renaming the base, or moving the function through the hierarchy, silently requires the expression to be updated. Worse, `std::declval`/`decltype` is an advanced idiom that obscures a trivial intent: *"return whatever the base returns."*

4.2 How common is this in practice?

This is not a hypothetical inconvenience. A survey of eight widely-used C++ codebases (LLVM, Godot, POCO, Catch2, spdlog, range-v3, fmt, and nlohmann/json) found that **2,523 of 61,474 single-line overriding declarations — about 4% — restate a verbose or dependent return type** that the language already fixes. That is thousands of real sites, rising to **7–8% in interface-heavy frameworks** (POCO, spdlog, Catch2). The same return type is routinely hand-copied across a whole family of subclasses: clang's `ArrayRef<TargetInfo::GCCRegAlias> getGCCRegAliases() const override` is restated verbatim in **25** target back-ends, and one Godot OpenXR return type, `HashMap<String, bool *>`, in **39**. POCO's database back-ends restate the same statement-factory return type in three *different* spellings of the one type (`Poco::SharedPtr<Poco::Data::StatementImpl>`, its unqualified form, and the typedef `StatementImpl::Ptr`), so a reader must chase a typedef just to confirm the override matches its base. The `decltype(std::declval<…>())` idiom shown above is itself real production code: LLVM's PDB unit tests wrap it in a macro precisely to avoid writing it out for each of scores of accessor overrides.

The hand-synchronisation this redundancy demands **already fails in practice**. In Godot, the base class and eight display-server back-ends declare `Vector<DisplayServerEnums::WindowID> get_window_list() const`, but the macOS back-end declares the *same* override as `Vector<int>` — a silent divergence that compiles only because `WindowID` happens to be an alias for `int`. Under this proposal every back-end writes `auto get_window_list() const override` and the inconsistency cannot arise. (These counts and examples were gathered by a heuristic text scan cross-checked with a Clang AST pass, and every figure is backed by openable `file:line` citations.)

4.3 What this paper proposes

Allow a virtual override to be written with the bare placeholder type `auto` and no trailing-return-type:

```
struct B : A {
  auto test() override { return 6; }
};
```

When a function is declared this way, its return type is determined to be **the return type of the unique function it overrides** in a direct or indirect base class. The determination happens at the point of declaration from the override relationship — it does **not** depend on the function body — so the return type is fixed wherever the class is complete, exactly as if it had been written out by hand.

The function body does not change the return type, but it **is** checked against it: every `return` statement must already yield that exact type. A `return` whose operand has a *different* type — even one that would implicitly convert — is ill-formed, not silently converted (see [design.returns]). This keeps the feature a pure abbreviation of the type while refusing to hide the kind of accidental conversion (`const char*` → `bool`, `int` → `long`) that an explicitly written-out return type would accept silently.

4.4 Why this is safe and small

The proposal deliberately ties the feature to the `override` specifier (see [design.trigger]). The `override` keyword already makes the program ill-formed unless the function overrides a base virtual function ([class.virtual]/4), so by construction there is always exactly one return type to copy. The function-matching machinery is therefore unchanged: a function marked `override` that fails to override anything is ill-formed today, and remains ill-formed under this proposal.

The proposal introduces exactly one new diagnosable situation, and it does so on purpose: a `return` statement whose operand does not already have the overridden return type ([design.returns]). This is a deliberate safety

choice — the alternative, silently converting the operand to the overridden type, is discussed and rejected in [design.returns].

5 Design discussion

5.1 The feature is triggered only by `override`

Deduction applies only to a function declared with **both** the placeholder type `auto` **and** the `override` specifier. We do *not* extend it to any function that merely happens to match a base virtual signature.

Tying the feature to `override` has three benefits:

- **A return type always exists to copy.** `override` guarantees a matching base function, so the deduction can never silently fail to find a source.
- **Intent is explicit.** The reader sees `override` and knows the function is bound to a base contract; the return type following from that contract is unsurprising.
- **No accidental coupling.** A non-`override` function with a deduced return type continues to deduce from its body, exactly as today. Adding or removing a base virtual function never silently changes the meaning of an unrelated `auto` function.

5.2 The type comes from the override, fixed at the declaration

The return type is the overridden function's return type, and it is **determined from the override relationship alone**, at the point of declaration — never from the body. This is what makes the feature compatible with the existing prohibition on virtual placeholder return types (see [design.prohibition]): the type is known wherever the class is complete, so vtable layout and override checking proceed exactly as today. Two consequences:

- A declaration with no in-class definition is fully supported, because the type is already fixed by the override:

```
struct B : A {
  auto test() override;     // return type is A::test's type: int
};
int B::test() { return 6; } // out-of-line definition; type already fixed
```

- **Covariance is not synthesized.** Today an override of `Base* f()` may return `Derived*`. Under this proposal, `auto f() override` yields exactly `Base*`. A programmer who wants a covariant return type must still write it explicitly:

```
struct A { virtual A* clone(); };
struct B : A {
  auto  clone() override;   // returns A*  (base type, exactly)
  B*    clone2() override;  // covariant return type, written out (unchanged today)
};
```

This keeps the rule trivially simple — *the override's type is the overridden function's type* — and preserves covariance as an explicit, opt-in act rather than something the compiler infers.

5.3 Return statements must already have the overridden type

Because `auto` everywhere else in the language deduces a function's return type *from its body*, the natural question — raised in early review — is what happens when the type a `return` statement would yield differs from the overridden type. Consider:

```
struct A { virtual bool f() = 0; };
struct B : A {
  auto f() override { return "pumpkin"; }   // body yields const char*, override says bool
};
```

There were two candidate answers:

1. **Silently convert.** Treat the declaration as pure shorthand for `bool f() override`, so `return "pumpkin"` undergoes the usual `const char*` → `bool` conversion (always `true`). Zero new behavior, but it preserves a sharp footgun: a pointer silently becoming `true`, an `int` silently widening to `long`, all hidden behind `auto`.
2. **Require an exact match.** Make the program ill-formed unless every `return` statement already yields the overridden type, forcing the author to convert explicitly.

This paper adopts **(2)**. The rule is: let `T` be the return type of the function being overridden. The return type of the function is `T`, and for each `return` statement, the type that `auto` would deduce from that statement ([dcl.spec.auto]) shall be `T`; otherwise the program is ill-formed. No implicit conversion to `T` is performed.

```
struct A { virtual long f() = 0; };
struct B : A {
  auto f() override { return 0; }                 // ill-formed: 0 is int, not long
  auto g() override { return 0L; }                // OK
  auto h() override { return static_cast<long>(x); } // OK: explicit cast
};
```

Equivalently — and this is the framing that makes it feel principled rather than special — the overridden type behaves as **one additional source of deduction**, exactly as if an extra `return` of type `T` were present. The existing rule that a function with multiple `return` statements of differing deduced type is ill-formed ([dcl.spec.auto]) then does all the work, and `auto f() override` is *not* a second, divergent meaning of `auto` so much as ordinary deduction with one extra, authoritative source. The cost is one new diagnostic and a little more typing at genuine type mismatches; the benefit is that the abbreviation never hides a conversion. Implementations are encouraged to phrase the diagnostic in terms of the overridden function (e.g. *"return type `int` does not match `long A::f()`"*) rather than as an ambiguous deduction.

5.4 Out-of-line definitions and mixed spellings

The return type of an overriding function is fixed by the override regardless of how any one declaration spells it. The proposal therefore lets `auto` and the written-out type be used interchangeably across the declaration and an out-of-line definition of the same overriding function, and makes a *disagreeing* written-out type ill-formed. All four combinations are permitted, with the obvious constraint:

```
struct A { virtual int f() = 0; };

struct B : A {
  // 1) auto declaration, auto definition
  auto f() override;
};
auto B::f() { return 3; }          // OK: deduces int (the overridden type)

struct C : A {
  // 2) auto declaration, written-out definition
  auto f() override;
};
int  C::f() { return 3; }          // OK: int matches A::f
// long C::f() { return 3; }       // ill-formed: long does not match A::f's int

struct D : A {
  // 3) written-out declaration, auto definition
  int  f() override;
};
auto D::f() { return 3; }          // OK: auto denotes the overridden type, int

struct E : A {
  // 4) written-out declaration, written-out definition  (status quo)
  int  f() override;
};
int  E::f() { return 3; }          // OK
```

This deliberately relaxes the existing rule that a redeclaration of a function with a placeholder return type must repeat the placeholder ([dcl.spec.auto.general]). The relaxation is safe precisely because, for an overriding function, the type is not actually being *deduced* across translation units — it is fixed by the base, so every spelling (`auto` or the written-out `T`) denotes the same known type, and a mismatch is a local, immediately diagnosable error.

5.5 Ambiguity across multiple base functions

A function may override more than one base function under multiple inheritance. If the overridden functions do not all have the same return type, there is no single type to copy and the program is ill-formed:

```
struct A { virtual int  f(); };
struct C { virtual long f(); };
struct D : A, C {
  auto f() override;  // ill-formed: A::f returns int, C::f returns long
};
```

Note this case is already ill-formed today for a different reason (an override cannot simultaneously satisfy two incompatible base return types), so the proposal does not introduce a new diagnosable situation here; it only specifies which diagnostic applies.

5.6 `decltype(auto)` and other placeholders

This paper addresses only the bare placeholder `auto`. A virtual function declared with `decltype(auto)` or a constrained placeholder (e.g. `std::integral auto`) remains ill-formed, as today. Extending the feature to those spellings is possible future work (see [future]) but is intentionally out of scope to keep the rule minimal.

5.7 Relationship to the existing prohibition on virtual `auto`

Today, a function whose declared return type uses a placeholder type "shall not be declared `virtual`" — a deduced-from-body return type is incompatible with the vtable model because the type must be known to all translation units that see the class definition. This proposal does **not** weaken that rationale: the return type of an `auto … override` function is known from the class definition (it is the base's return type, which is visible wherever the derived class is complete), so every translation unit agrees on the type without seeing the body. The proposal therefore carves a narrow, well-defined exception out of the existing prohibition rather than removing it.

5.8 Interaction with `final`, access, `noexcept`, and qualifiers

`final`, access specifiers, `noexcept`, ref-qualifiers, and cv-qualifiers are orthogonal to the return type and are unaffected. `auto f() const override`, `auto f() && override`, `auto f() noexcept override` all behave as expected: the signature determines which base function is overridden, and that function's return type is copied.

6 Proposed wording

The proposed changes are relative to the C++ working draft [N5046]. Existing standard text is quoted verbatim; inserted text is shown like this and ~~removed text like this~~. Paragraph numbers match N5046 and are to be adjusted editorially. Three paragraphs/tables are affected.

6.1 Change 1 — relax the virtual-function prohibition

In **[dcl.spec.auto.general]** paragraph 17, the placeholder return type is forbidden on *every* virtual function. The current text reads, in its entirety:

17 A function declared with a return type that uses a placeholder type shall not be virtual ([class.virtual]).

Modify it to permit the bare-`auto` + `override` form:

17 A function declared with a return type that uses a placeholder type shall not be virtual ([class.virtual])~~.~~ unless the placeholder type is the `auto` *type-specifier* not followed by a *trailing-return-type* and the

function overrides a virtual function and is declared with the `override` *virt-specifier* on its first declaration ([class.virtual]); in that case the return type of the function is the return type of the function it overrides, as specified in [class.virtual]. [Note: The return type of such a function is therefore not deduced from its `return` statements; instead, each `return` statement is checked against that return type ([class.virtual]). — end note]

Drafting note. No other paragraph of [dcl.spec.auto.general] needs to change: paragraph 8 ("a program that uses a placeholder type in a context not explicitly allowed … is ill-formed") already defers to the per-context rules, and this is the only context that forbade a virtual function.

6.2 Change 2 — specify the return type from the overridden function

In **[class.virtual]**, the override-matching rule (paragraph 2) keys on the *name* and *parameter-type-list* and never on the return type, and paragraph 5 makes `override` ill-formed unless the function overrides something. Add a new paragraph immediately after paragraph 8 (the covariant-return-type rule) to determine the return type in the new case. For reference, paragraph 8 currently begins:

8 The return type of an overriding function shall be either identical to the return type of the overridden function or covariant with the classes of the functions. […]

Insert after it:

(8.*) If the declared return type of an overriding function `G` uses the `auto` placeholder type ([dcl.spec.auto]), then `G` overrides at least one function and is declared with the `override` *virt-specifier* on its first declaration ([dcl.spec.auto.general]). If the functions that `G` overrides do not all have the same return type, the program is ill-formed. Otherwise, let `T` be that common return type; the return type of `G` is `T`. For each *return statement* in the body of `G`, the type that would be deduced for `auto` from that *return statement* ([dcl.spec.auto]) shall be `T`; no implicit conversion to `T` is performed by virtue of this subclause. [Note: A `return` whose operand has a type other than `T` is therefore ill-formed even when that type is implicitly convertible to `T`; an explicit conversion is required. The return type `T` is identical to the return type of each overridden function and is therefore not covariant ([class.virtual]/8) with it; a covariant return type must be declared explicitly. — end note]

Drafting note. Because paragraph 2's override matching does not consult the return type, the set of overridden functions — and hence "that common return type" — is well-defined before `G`'s return type is known; there is no circularity.

6.3 Change 3 — allow `auto` and the written-out type to be mixed across declarations

In **[dcl.spec.auto.general]**, the rule that a redeclaration must repeat the placeholder otherwise prevents an out-of-line definition from spelling the return type explicitly while the in-class declaration uses `auto`, or vice versa. For an overriding function the return type is fixed by the base ([class.virtual]), so this restriction can be relaxed. The current text reads (paragraph number provisional):

13 Redeclarations or specializations of a function or function template with a declared return type that uses a placeholder type shall also use that placeholder, not a deduced type.

Modify it:

13 Redeclarations or specializations of a function or function template with a declared return type that uses a placeholder type shall also use that placeholder, not a deduced type~~.~~, except that a declaration of a function that overrides a virtual function ([class.virtual]) may use either the `auto` *type-specifier* without a *trailing-return-type* or the return type of the overridden function, in any combination. If two declarations of such a function specify return types that are not the same type, the program is ill-formed.

Drafting note. This permits all four combinations of `auto` / written-out return type across the in-class declaration and an out-of-line definition of an overriding function (see [design.outofline]); a written-out type that disagrees with the overridden type is ill-formed and diagnosed locally.

6.4 Change 4 — feature-test macro

In **[cpp.predefined]**, add a row to Table [tab:cpp.predefined.ft] ("Feature-test macros"), in alphabetical order among the other `__cpp_` macros:

Macro name	Value
`__cpp_deduced_virtual_override_return`	`202606L`

Drafting note. The value `202606L` is provisional; the editor assigns the year-and-month (`YYYYMML`) of the meeting at which the feature is adopted. The name uses "virtual" because the feature applies precisely to virtual functions: `override` is permitted only on a function that overrides a base virtual function ([class.virtual]/5), which is therefore itself virtual.

6.5 Summary of edits

#	Stable label	Location	Edit
1	[dcl.spec.auto.general]	para 17	Append an exception permitting `auto` on an overriding function declared `override`; point to [class.virtual] for the resulting type.
2	[class.virtual]	new para, after 8	Fix the return type to the overridden function's; make differing return types across multiple overridden functions ill-formed; require each `return` statement to already yield that exact type (no implicit conversion).
3	[dcl.spec.auto.general]	redeclaration para (~p13)	Exempt overriding functions from the "repeat the placeholder" rule, so `auto` and the written-out type may be mixed across declarations; disagreeing written-out types are ill-formed.
4	[cpp.predefined]	Table [tab:cpp.predefined.ft]	Add the `__cpp_deduced_virtual_override_re` feature-test macro.

7 Impact on the Standard

This is a pure language extension. It:

- changes no existing valid program's meaning (code that does not combine `auto` with `override` is unaffected);
- makes well-formed a category of declarations that are ill-formed today (`auto f() override` with no trailing-return-type on a virtual function);

— requires no library changes;
— requires no changes to the ABI or object model — an `auto … override` function has the same type, mangling, and vtable slot it would have had if its return type were written out by hand.

There are no breaking changes and no deprecations.

8 Implementation experience

The feature requires no novel analysis. A conforming compiler already:

1. resolves which base function(s) a member overrides (to validate `override` and covariance), and
2. has the overridden function's return type available at that point.

Implementing the proposal amounts to: when a virtual function's declared return type is the bare `auto` placeholder and `override` is present, set the function's return type to the overridden function's return type (after the existing override resolution) instead of entering body-based deduction.

This has been confirmed by a prototype in Clang (against LLVM trunk). The implementation is a **156-line additive change to Sema**, gated behind `-std=c++2c`, and reuses machinery the compiler already runs — override resolution (which executes while the member is declared) and the existing `auto` return-type deduction. Concretely:

— the existing "different return type" diagnostic is *deferred* when an overriding function's declared return type is the bare `auto` placeholder;
— once the `override` specifier and the set of overridden functions are known, the function's return type is set to the overridden type and fixed at the declaration, so vtable layout and override checking proceed exactly as for a written-out type;
— each `return` statement is checked against that type with no implicit conversion;
— mixing `auto` and the written-out type across an in-class declaration and an out-of-line definition is reconciled in the existing redeclaration-merging step, with a disagreeing written-out type diagnosed; and
— a feature-test macro `__cpp_deduced_virtual_override_return` is defined.

The prototype passes a positive/negative conformance suite covering every case in this paper — the basic form, all four `auto`/written-out out-of-line combinations, a dependent return type, cv- and `noexcept`-qualified overrides, and each ill-formed case (return-operand mismatch, missing `override`, `decltype(auto)`, disagreeing multiple bases, and a disagreeing out-of-line definition) — together with an in-tree `-verify` test. Existing override, covariance, and ordinary `auto`-deduction behaviour is unchanged, and the same spelling remains ill-formed before C++26. The branch is available at *github.com/Alex-Tsvetanov/llvm-project* (branch `auto-return-override`).

9 Future directions

The following are intentionally **out of scope** for this paper but are natural follow-ups:

— **`decltype(auto)` overrides.** Could be defined to copy the base type with its value category, though the practical benefit over `auto` is small.
— **Deduced covariant return types.** Allowing `auto` to deduce a covariant type from the body (rather than copying the base type exactly) was considered and rejected for R0 in favor of a simpler rule; it could be revisited.
— **Dropping the `override` requirement.** Permitting deduction for any overriding function was rejected (see [design.trigger]); revisiting it would require a story for silent overrides and accidental coupling.

10 Acknowledgements

To be added.

11 References

12 References

[N5046] 2026-05-12. Working Draft, Programming Languages — C++.
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2026/n5046.pdf